Güvenli Kodlama ve Programlama

Zafiyet Analiz

Dönem Projesi Yönergesi

Bireysel çalışma · Dönem notuna katkı: %20 · Video: 10–15 dakika

1 Amaç ve kapsam

Bu ödevin amacı, yayımlanmış bir güvenlik açığının nedenini, etkisini ve giderilme yöntemini uygulamalı olarak incelemektir. Öğrenciden, seçtiği zafiyet için bir laboratuvar (lab) ortamı kurması, hatayı yeniden üretmesi ve düzeltme sonrasında elde edilen sonucu açıklaması beklenmektedir. Çalışma bireysel olarak yürütülecektir. Web, masaüstü ve mobil uygulamalar, API'ler, embedded sistemler, IoT cihazları ve firmware bileşenleri kapsam dahilindedir.

2 Konu seçimi

Seçim önceliği: Konu seçiminde mümkün olduğunca yaygın kullanılan, popüler ürünlere veya bilinen güvenlik vakalarına öncelik verilmelidir. Tarih sınırı yoktur; eski konular da popüler bir ürüne veya bilinen bir vakaya ilişkin olması hâlinde seçilebilir.

Aşağıdaki iki yaklaşımdan biri seçilebilir:

a) **Gerçek ürün üzerinde CVE incelemesi.** Zafiyetli sürümü kurulabilen bir yazılım seçilir. CVE, yayımlanmış bir güvenlik açığının kayıt numarasıdır. İlgili zafiyet lab ortamında yeniden üretilir; ilgili kod veya yazılım davranışı ile düzeltme incelenir.

b) **Yayımlanmış bir vakanın incelenmesi.** Facebook, Instagram veya benzeri bir uygulamada açıklanmış bir güvenlik açığı ele alınabilir. Kaynak koduna erişilemiyorsa aynı hata mantığını içeren bir temsili lab hazırlanabilir; isteğe bağlı olarak reverse engineering ile ilgili yazılım da incelenebilir. Bu yaklaşımda CVE numarası aranmaz; vakayı açıklayan bir kaynak gösterilmesi gerekir.

Konu bildirimi: Seçilen konu `https://forms.gle/BWrEbR3Rkev5CSZb7` adresindeki formla bildirilir; CVE yoksa vakayı anlatan kaynağın bağlantısı forma yazılır. Sorular ve konunun uygunluğu için `canoztas.sektorkampuste@gmail.com` adresine yazılabilir.

3 Uygulama aşamaları

1. **Lab kurulumu:** Docker, sanal makine, yerel uygulama, mobil emülatör veya firmware emülasyonu kullanılabilir. Öğrencinin kendi cihazıyla izole ortamda çalışması veya temsili lab hazırlaması da mümkündür. Sürümler ve kurulum adımları kaydedilmelidir.
2. **Zafiyetin yeniden üretilmesi:** Önce normal kullanım, ardından hatayı tetikleyen işlem gösterilmelidir. Beklenen ve gözlenen sonuçlar açıklanmalıdır.
3. **Nedenin incelenmesi:** Hataya yol açan kod, eksik kontrol veya yanlış varsayım belirlenmeli; güvenlik etkisiyle ilişkisi açıklanmalıdır.
4. **Düzeltmenin uygulanması:** Gerçek üründe yayımlanan yama incelenip uygulanmalı veya reverse engineering ile bir fix geliştirilmelidir. Temsili uygulamada uygun düzeltme geliştirilmelidir.
5. **Sonucun doğrulanması:** Aynı deneme düzeltmeden sonra tekrarlanmalı; zafiyetli davranışın giderildiği ve normal işlevin korunduğu gösterilmelidir.

Lab kurulumu ve uygulamalı gösterim zorunludur. Tüm denemeler öğrencinin kendi yerel veya izole ortamında, sahte hesap ve verilerle yapılmalıdır. Üçüncü taraflara ait canlı sistemlerde deneme yapılmamalıdır.

4 Örnek konular

Aşağıdaki konu havuzu, yaygın kullanılan ürünleri ve bilinen platformlara ilişkin yayımlanmış vakaları kapsar. Öğrenci, bu ölçütleri karşılayan, kurulabilir ve incelenebilir başka bir konu da seçebilir. Her örnekte olayın hikâyesi, teknik odak ve önerilen lab kapsamı verilmiştir.

4.1 Ürün ve framework örnekleri

Tablo 1: Ürün zafiyetleri ve önerilen lab kapsamları.

Ürün / CVE	**Olay ve teknik odak**	**Önerilen lab**
Apache HTTP Server CVE-2021-41773 [1]	2021'de path normalization hatası, belirli konfigürasyonlarda web kökü dışındaki dosyalara erişime yol açtı. İlk fix'in eksik kalması, patch analizi için de bir örnektir.	2.4.49 ve 2.4.51 sürümlerini sahte bir dosyayla karşılaştırın. 2.4.50'deki ilk fix'in neden yeterli olmadığını açıklayın.
Django CVE-2022-34265 [2]	ORM kullanımı tek başına SQL injection'ı engellemiyordu: Trunc / Extract işlevlerindeki doğrulanmamış tarih bölümü girdisi SQL sorgusunu etkileyebiliyordu.	3.2.13 ve 3.2.14 ile küçük bir uygulama kurun. İlgili girdinin sorguya nasıl ulaştığını ve fix'in neyi reddettiğini gösterin.
Grafana CVE-2021-43798 [3]	2021'de dashboard platformunun plugin dosyalarını sunan yolu, path traversal nedeniyle uygulama klasörü dışındaki dosyaları okumaya izin veriyordu.	8.3.0 ve 8.3.1'i karşılaştırın. Lab için oluşturduğunuz bir dosyayla hatayı ve path kontrolündeki fix'i gösterin.
Apache Log4j CVE-2021-44228 [4]	Log4Shell: Java loglama kütüphanesine verilen girdinin JNDI lookup olarak işlenmesi, uygun koşullarda remote code execution'a yol açabiliyordu.	Küçük bir Java uygulamasında yerel JNDI çağrısını gözlemleyin; fix sonrası davranışla karşılaştırın. Java sürümünü ve gerekli konfigürasyonu belgeleyin.
Next.js CVE-2025-29927 [5]	2025'te, authorization kontrolü middleware içinde yapılan uygulamalarda özel bir HTTP header ile kontrolün atlanabildiği açıklandı.	Yerel sunucuda 14.2.24 ve 14.2.25 ile korumalı bir sayfayı karşılaştırın. Fix sonrası aynı isteğin reddedildiğini gösterin.
GitLab CE/EE CVE-2023-7028 [6]	2024'te açıklanan hata, password reset e-postasının doğrulanmamış bir adrese gönderilmesine izin veriyordu. 2FA olmayan hesaplarda account takeover mümkündü.	16.7.1 ve 16.7.2'yi sahte hesaplar ve yerel e-posta yakalayıcıyla karşılaştırın. Reset alıcısını ve 2FA'nın etkisini açıklayın.
Jenkins CVE-2024-23897 [7]	2024'te CLI argument parser'ın dosya içeriğini argümana dönüştürme özelliği, sunucuda file disclosure'a neden oldu. Okunabilen içerik yetkilere göre değişiyordu.	LTS 2.426.2 ve 2.426.3'ü sahte metin dosyasıyla karşılaştırın. Kullanıcı yetkisini ve parser fix'ini inceleyin.

Belirtilen sürümler yalnızca ilgili zafiyeti karşılaştırmak içindir. Lab kurulmadan önce kaynakta açıklanan konfigürasyon ve önkoşullar kontrol edilmelidir.

4.2 Yayımlanmış hack vakaları ve araştırmalar

Bu örnekler gerçekleşmiş güvenlik olayları ile araştırmacıların yayımladığı bulguları içerir. Seçilen vakada ne yaşandığı, zafiyetin nasıl keşfedildiği ve teknik nedenleri rapor ve videolu sunumda anlatılmalıdır.

Facebook: "View As" ve access token sızıntısı (2018) [8]

Facebook, gerçek saldırıda kullanılan üç hatanın birleşimini açıkladı: yalnızca önizleme sunması gereken View As ekranında video yükleme bileşeni çalışıyor, yanlış kullanıcı için access token üretiyor ve bunu HTML içinde açığa çıkarıyordu. Saldırganlar bu token'larla başka kullanıcıların hesabına erişebiliyordu.

Lab fikri: İki sahte hesaplı profil önizlemesi ve token üreten bir bileşen yazın. Kullanıcı bağlamının karışmasını gösterin; token üretimi ve authorization kontrollerini düzeltin.

Instagram: hesap kurtarmada rate limit bypass (2019) [9]

Laxman Muthiyah, mobil hesap kurtarma akışındaki altı haneli kodun deneme sınırını araştırdı. Eşzamanlı istekler ve farklı IP'ler, mevcut rate limit'in beklenenden fazla denemeye izin vermesine yol açıyordu. Araştırmacı bulguyu Facebook'a bildirdi ve fix uygulandı.

Lab fikri: Kodla password reset yapan küçük bir uygulama kurun. Birden fazla istemciyle deneme sınırını inceleyin; hesap bazlı ortak sayaç ve atomik kontrol ile fix'i gösterin.

Philips Hue Bridge: Zigbee üzerinden firmware zafiyeti (2020) [10]

Check Point araştırmacıları, akıllı ampulü taklit ederek Hue Bridge'in Zigbee mesajlarını işleme biçimini inceledi. Eşleştirme sırasında gönderilen uzun bir ZCL string'i, firmware'de heap-based buffer overflow'a ve bridge üzerinde kod çalıştırmaya yol açıyordu. CVE-2020-6007 için üretici bir firmware fix'i yayımladı.

Lab fikri: Zigbee mesajı işlemeyi temsil eden küçük bir parser veya emüle firmware bileşeni kurun. Uzunluk kontrolündeki hatayı ve fix'i sentetik verilerle gösterin; gerçek vakayla ilişkisini açıklayın.

YouTube: TV özelliğinden private videolara erişim (2021) [11]

David Schütz, hesabıyla giriş yapmadığı bir TV'nin kendi private videosunu nasıl oynattığını merak ederek araştırmaya başladı. Yayımladığı vakada, TV oynatma akışındaki eksik CSRF kontrolü kullanıcının video erişim token'ının araştırmacının taklit ettiği TV'ye aktarılmasına izin veriyordu.

Lab fikri: Sahte videolar, kullanıcı oturumu ve temsili TV istemcisiyle bir paylaşım akışı kurun. CSRF nedeniyle istenmeyen token aktarımını ve erişim kontrolüyle giderilmesini gösterin.

TikTok Android: deep link ve WebView zinciri (2022) [12]

Microsoft araştırmacıları, deep link doğrulamasını aşarak WebView içinde güvenilmeyen bir URL açılabildiğini gösterdi. JavaScript bridge erişimiyle hesap üzerinde işlem yapılabiliyordu. CVE-2022-28799 olarak kaydedilen sorun TikTok tarafından düzeltildi; Microsoft aktif kötüye kullanım kanıtı bulmadığını belirtti.

Lab fikri: Emülatörde küçük bir Android uygulaması, WebView ve yerel test sayfası kullanın. URL doğrulamasını ve bridge yetkilerini sınırlandırarak fix'i gösterin.

Google Pixel: SIM PUK akışından lock screen bypass (2022) [13]

David Schütz, unuttuğu SIM PIN'ini PUK ile sıfırlarken telefonun kilit ekranında beklenmeyen davranış gördü. İnceleme, fiziksel erişimle cihaz kilidinin aşılabildiğini ortaya koydu. CVE-2022-20465, Kasım 2022 güvenlik güncellemesinde giderildi; Android fix'inin kaynak kodu yayımlandı.

Lab fikri: SIM doğrulaması ile cihaz kilidini ayrı state'ler olarak modelleyen küçük bir uygulama yazın. Yanlış ekranın kapatılmasını ve state kontrolüyle fix'i gösterin; Android patch'iyle ilişkilendirin.

5 Kapalı kaynak yazılımlar ve yapay zekâ kullanımı

Kapalı kaynak bir uygulama için, yalnızca incelenen vakayla ilgili işlevi içeren bir temsili lab hazırlanması yeterlidir. Basit bir arayüz ve birkaç sahte hesap kullanılabilir.

Reverse engineering isteğe bağlıdır. Öğrenci bu yöntemle yazılımın davranışını inceleyebilir, kendi lab ortamında geliştirme veya fix yapıp sonucu test edebilir. Temsili lab hazırlamak da geçerli bir seçenektir.

Uygulama, yapay zekâ destekli kodlama araçlarıyla (*vibe coding*) geliştirilebilir. Kullanılan araçlar ve alınan desteğin kapsamı raporda belirtilmelidir. Öğrenci, üretilen kodu açıklayabilmeli; hatalı ve düzeltilmiş davranışı kendi ortamında doğrulamalıdır.

Temsili uygulama kullanılan çalışmalar *temsili lab* olarak adlandırılmalı; gerçek vakayla benzerlikler, sadeleştirmeler ve kullanılan düzeltme açıklanmalıdır.

Örnek konuların kaynakları

[1] Apache HTTP Server Project. *Apache HTTP Server 2.4 vulnerabilities.*

[2] Django Project. *Django 3.2.14 release notes.*

[3] Grafana Labs. *Grafana 8.3.1, 8.2.7, 8.1.8, and 8.0.7 security fix.*

[4] Apache Logging Services. *Log4j security advisories: CVE-2021-44228.*

[5] Vercel / Next.js. *Authorization Bypass in Next.js Middleware.*

[6] GitLab. *Critical Security Release: 16.7.2, 16.6.4, 16.5.6.*

[7] Jenkins Project. *Security Advisory 2024-01-24.*

[8] Meta. *Security Update: View As.*

[9] Laxman Muthiyah. *How I Could Have Hacked Any Instagram Account.*

[10] Check Point Research. *Don't be silly: it's only a lightbulb.*

[11] David Schütz. *I Built a TV That Plays All of Your Private YouTube Videos.*

[12] Microsoft Security. *Vulnerability in TikTok Android app could lead to one-click account hijacking.*

[13] David Schütz. *Accidental $70k Google Pixel Lock Screen Bypass.*

6 Teslim edilecek çalışmalar

6.1 Rapor

Rapor PDF biçiminde hazırlanmalıdır; **sayfa sınırı yoktur.** Öğrencinin adı, numarası ve seçilen konu belirtilmelidir. Zafiyet, konuyu önceden bilmeyen bir okuyucunun anlayabileceği ayrıntıda açıklanmalıdır:

- **Zafiyet nedir?** Ürün veya vaka, etkilenen işlev ve sürümler, zafiyetin türü, oluşma koşulları ve kaynakları;
- **Neden oluşur ve etkisi nedir?** Root cause, hatalı kod veya eksik kontrol; saldırganın kontrol ettiği girdi, gereken erişim ve olası güvenlik etkisi;
- **Nasıl yeniden üretilir?** Lab kurulumu, sürümler, normal kullanım ve zafiyeti tetikleyen adımlar; beklenen ve gözlenen sonuçlar;
- **Nasıl düzeltilir?** Yama veya fix'in neyi değiştirdiği, neden işe yaradığı ve aynı deneme ile normal kullanım testlerinin düzeltme sonrası sonuçları.

Kod parçaları ve ekran görüntüleri, neyi kanıtladıklarını açıklayan notlarla sunulmalıdır. Temsili lab kullanılmışsa gerçek vakayla ilişkisi; reverse engineering kullanılmışsa yöntem ve yapılan değişiklikler açıklanmalıdır.

6.2 Anlatım videosu

10–15 dakikalık bir ekran kaydı hazırlanmalıdır. Videolu sunumda zafiyetin ne olduğu, etkilenen işlev, oluşma koşulları, root cause ve güvenlik etkisi ayrıntılı anlatılmalıdır. Öğrenci kendi sesiyle labı, hatanın oluşumunu, fix'i ve doğrulama testlerini açıklayarak göstermelidir. Kamera zorunlu değildir; kurulumdaki bekleme süreleri çıkarılabilir.

Video, YouTube'da liste dışı veya eğitmen erişimi önceden tanımlanmış gizli video olarak ya da Google Drive bağlantısıyla paylaşılabilir. **Bağlantı doğrudan erişilebilir olmalı; "erişim iste" ekranı veya ek izin onayı gerektirmemelidir.** Erişim ayarları teslimden önce kontrol edilmelidir.

6.3 Lab dosyaları

Kaynak kodu, gerekli konfigürasyonlar ve testler sunulmalıdır. `README.txt` veya `README.md`; sürümleri, kurulum ve çalıştırma komutlarını, deneme adımlarını ve beklenen sonuçları içermelidir. Hatalı ve düzeltilmiş kod ayrı klasör veya sürümlerde gösterilmelidir.

GitHub reposu kullanılması tavsiye edilir. Repo; kod, README, rapor ve video bağlantısıyla CV'de paylaşılabilecek ve teknik görüşmelerde gösterilebilecek bir portföy projesi olabilir. GitHub kullanımı isteğe bağlıdır; repo eğitmen tarafından doğrudan erişilebilir olmalıdır.

GitHub kullanılmıyorsa dosyalar `ogrenciNo_konu.zip` olarak teslim edilebilir. Bağımlılık klasörleri ve büyük sanal makine dosyaları eklenmemelidir.

7 Değerlendirme

Çalışma 100 puan üzerinden değerlendirilecek ve dönem notuna **%20** oranında katkı sağlayacaktır. Gerçek ürün incelemesi ve temsili lab aynı ölçütlerle değerlendirilecektir.

Tablo 2: Değerlendirme ölçütleri.

Ölçüt	**Puan**
Konu, etki ve kaynakların açıklığı	10
Zafiyetin nedeninin kod veya kontrol üzerinden açıklanması	25
Lab kurulumu ve zafiyetin yeniden üretilmesi	20
Düzeltmenin açıklanması ve testlerle doğrulanması	25
Raporun, kurulum notunun ve kaynak gösteriminin niteliği	10
Videonun açıklığı ve uygulamalı anlatım	10
Toplam	**100**

Takvim ve teslim: Konu formla bildirilir (bkz. 2. bölüm); teslim tarihleri ve kanalı duyurulacaktır.